

Deep Learning for Multi-Scale Time Series Forecasting

Stefano Mauloni, Raffaele Simbari, Andrea Lisco, Alessandro Bottardi
Bocconi University

Spring 2025

1 Introduction

This report describes the methodology and results of two forecasting tasks: daily predictions of the EURGBP exchange rate and 5-minute-level prediction of the NASDAQ100 index. Using data from *LSEG Workspace*, licensed by our institution, we construct a feature-rich dataset by computing standard technical indicators, denoise the dataset, and use different techniques to infer the next values of the series. Models involved are both statistical (ARIMA with GARCH effects) and Deep-Learning based (Long-Short term memory networks with different dense output layers). To improve model robustness and capture different aspects of the predictive distribution, we implement hyperparameter tuning over several loss functions and compare their performance. In what follows, we describe the data preprocessing steps, denoising techniques, model architecture, and hyperparameter search strategy.

2 Data and Feature Engineering

2.1 Raw Data Sources

The primary target variables are the daily closing value of the EURGBP exchange rate, spanning January 1996 through today, and NASDAQ100 for intraday data, spanning from January 2025 to today. In addition to the exchange rate itself, we collect the following covariates on a synchronous daily basis:

- EURUSD and GBPUSD exchange rates
- Major equity indices: STOXX 50, FTSE 100, NASDAQ 100
- Implied volatility indices: VIX (CBOE Volatility Index), if available
- Commodity indicators: spot gold (XAU/USD), Brent crude oil (if feasible)

All series are aligned by date; missing values (e.g., non-trading days) are handled via forward-fill or linear interpolation, depending on series characteristics.

Similar variables are also collected for the intraday forecast (specifically: NASDAQ_100_NQc1, EURUSD, GBPUSD, SPX_ESc1, USDXAU=R, VIX, WTI). In that case, we used data from the future market (primarily the Chicago Mercantile Exchange (CME)), since prices reflect almost perfectly the ones on the stock exchange, but the market is tradable 23 hours a day.

In principle, one should also account for the expiry of the future contracts, which has an impact on the value of the future contract compared to the one of the underlying asset, but *LSEG Workspace* already provides data that are processed to account for that (the so-called *rolled* series).

2.2 Technical Indicator Computation

To augment the raw price and covariate signals, we compute a suite of standard technical indicators for some of the assets in both cases (as done e.g. in Wei Bao [2017]). These include:

- **Moving Averages:** Simple moving averages over 5 and 10-period lookback windows and exponential moving averages over 10 and 20-period lookback windows.
- **Momentum and Trend Strength Indicators:**
 - 6/12-months Momentum: $Price_t - Price_{t-6/12\text{ months}}$.
 - Stochastic Momentum Index: Momentum oscillator measuring the speed and change of price movements, derived from the Relative Strength Index.
 - Moving Average Convergence Divergence: Trend-following momentum indicator that shows the relationship between two exponential moving averages of prices.
- **Volatility Measures:** Bollinger Band Width: Difference between the upper and lower Bollinger Bands, which are usually set two standard deviations above and below a simple moving average.

Each technical indicator is computed using the raw price series.

2.3 Lagged Returns and Feature Windowing

For time-series models, it is important to have stationary series. To do so, we compute differences (or percentage change for equities) along time series.

Then, for each feature x_t , we include its values at time steps $t-1, t-2, \dots, t-\ell$, where ℓ is the maximum lookback horizon (i.e., 20). Thus, for a single indicator, the model sees a window $\{x_{t-\ell}, \dots, x_{t-1}\}$. In practice, we stack all lagged features into a three-dimensional tensor of shape $(N_{\text{train}}, \ell, d)$ for model input.

3 Scaling and Normalization

Before model training, all features are scaled to ensure numerical stability:

- **Training Set Scaling:** Using only the training subset, we compute per-feature mean μ_j and standard deviation σ_j .
- **Z-Score Transformation:** For each feature $x^{(j)}$, transform $x_{t,\text{scaled}}^{(j)} = \frac{x_t^{(j)} - \mu_j}{\sigma_j}$.
- **Validation/Test Consistency:** The same μ_j, σ_j computed on the training data are applied to transform validation and test features, preventing information leakage.

4 Denoising via Wavelet Transform

4.1 Motivation for Denoising

Financial time series often contain high-frequency noise that can degrade neural network training and lead to overfitting on spurious fluctuations. To isolate the underlying signal, we implement the wavelet transform technique to decompose the time series into components at different frequency levels, allowing for effective noise reduction (see e.g. Donoho [1995], Amin Karimi Dastgerdi [2022])

This transformation is applied only to the daily data. Applying it for the intraday prediction task would have likely resulted in a significant loss of signal, given the different nature of the data.

4.2 Discrete Wavelet Transform (DWT) Workflow

For each univariate series y_t :

1. Choose an orthonormal wavelet basis.
2. Perform a multilevel DWT up to level L (i.e. $L = 2$), decomposing y_t into detail coefficients $\{d_{j,k}\}$ and approximation coefficients $a_{j,k}$.
3. Apply thresholding to detail coefficients at each level. We use a universal threshold $\sigma\sqrt{2\log(n)}$, where σ is estimated from the median absolute deviation (MAD) of the highest-level details, and n is the series length.
4. Reconstruct the denoised signal $\tilde{y}_t$ via inverse DWT using the thresholded coefficients.

After denoising, we obtain $\tilde{y}_t$ for each original covariate series and indicator. These filtered signals replace the raw values in the input tensor. We observe that high-frequency “spikes” are significantly attenuated, while major trends and turning points remain intact.

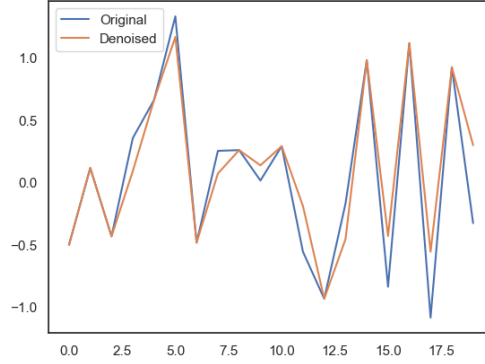


Figure 1: Example of denoised signal

5 Baseline model: ARIMAX

Our baseline model is an AutoRegressive Integrated Moving Average with eXogenous variables (ARIMAX), a classical autoregressive model used in time series forecasting.

The ARIMAX(p, d, q) model with exogenous variables can be written as:

$$\Phi(B)(1 - B)^d y_t = \Theta(B)\varepsilon_t + \beta(B)x_t$$

where:

- y_t is the dependent time series,
- x_t is the exogenous vector of variables,
- B is the backshift operator, i.e., $By_t = y_{t-1}$,
- $\Phi(B) = 1 - \phi_1 B - \phi_2 B^2 - \dots - \phi_p B^p$ is the AR polynomial,
- $\Theta(B) = 1 + \theta_1 B + \theta_2 B^2 + \dots + \theta_q B^q$ is the MA polynomial,
- $\beta(B) = \beta_0 + \beta_1 B + \dots + \beta_r B^r$ is a finite distributed lag polynomial for the exogenous input,

- ε_t is a white noise error term,
- d is the order of differencing to achieve stationarity.

In our case, $d = 0$ since we have already applied first differences to make the series stationary. We further check it using the Augmented Dickey-Fuller (ADF) test, a common test for stationarity.

We should remark that this model has been used only on the daily forecast of EURGBP. Applying this model to the intraday data resulted in a constant prediction of the mean. However, this is expected, since correlations among features are very complex in that timeframe, and a relatively simple model like ARIMAX is not expected to capture them.

6 Model Architecture: LSTM with Dense Output

6.1 Overall Network Structure

Our core model is a Long Short-Term Memory (LSTM) network followed by a fully connected output layer.

LSTMs are a type of Recurrent Neural Network (RNN) designed to model sequential data while mitigating issues like vanishing gradients. They are especially suited for tasks requiring long-term dependencies and contextual memory. The primary function of an LSTM is to learn patterns in sequences and make predictions based on temporal context.

An LSTM unit maintains two states: a cell state C_t (long-term memory) and a hidden state h_t (short-term output). Information flow is regulated through three gates:

- **Forget Gate:** Decides what to remove from the previous cell state.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

This output is in $[0, 1]$ and represents what to keep and what to discard.

- **Input Gate:** Determines which new information to add.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

While a tanh layer creates new candidate values

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

- **Cell State Update:** These are combined to update the state.

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

Specifically, the old state is multiplied by the output of the forget layer f_t , which means it forgets what we decided to forget, and the input values are scaled by how much it has been decided to update each state value. The two are finally added.

- **Output Gate:** Selects the output and updates the hidden state.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \cdot \tanh(C_t)$$

Let $\mathbf{X}_{t-\ell+1:t} \in \mathbb{R}^{\ell \times d}$ denote the lookback window of scaled features for day t . The network processes this tensor as follows:

$$\mathbf{h}_t = \text{LSTM}(\mathbf{X}_{t-\ell+1:t}) \quad , \quad \hat{y}_{t+1} = \mathbf{W}_{\text{out}} \mathbf{h}_t^{(\text{last})} + \mathbf{b}_{\text{out}},$$

where:

- LSTM($\cdot$) denotes the recurrent layer(s) with hidden size H .
- $\mathbf{h}_t^{(\text{last})} \in \mathbb{R}^H$ is the last hidden state of the LSTM after processing the sequence.
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{p \times H}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^p$ form the dense layer that maps to p -dimensional vector. In our case, $p = 1$ or 2 .

6.2 Types of Regressions and Loss Functions

To capture different aspects of the predictive distribution and to mitigate the influence of outliers, we compare two different types of regressions. We call them '**standard**' and '**probabilistic**'. In the first case, the dense layer of the LSTM model outputs a single real value (point estimate), while in the second case, we get two values: the mean and the volatility of the prediction. This allows for a more informative prediction of the next value, incorporating risk-related information. For the 'Standard' and 'Probabilistic' models we used, respectively:

1. Smooth L_1 :

$$\mathcal{L}(e) = \begin{cases} \frac{1}{2}e^2, & |e| \leq 1, \\ |e| - \frac{1}{2}, & |e| > 1, \end{cases}$$

where $e = y_{\text{true}} - y_{\text{pred}}$. This loss is less sensitive to large errors than mean squared error, but differentiable at zero.

2. Negative Gaussian Log-Likelihood (NLL):

The network outputs both $\mu = \hat{y}_{t+1}$ and $\log \sigma$. The NLL per sample is

$$\mathcal{L}_{\text{NLL}} = \frac{1}{2} \left[\log(2\pi) + 2 \log \sigma + \frac{(y - \mu)^2}{\sigma^2} \right].$$

This approach allows the model to express heteroskedastic uncertainty.

6.3 Volatility Regimes (NASDAQ100)

For the NASDAQ100 prediction task, the training portion captures the biggest spikes, while validation and test follow afterward when volatility is more subdued. This is problematic since we want the training set to best resemble the validation and test sets.

For this reason, we split the data into two different volatility regimes (high and low) and train different models on each of them. We ensure no look-ahead bias is introduced by computing the threshold for assigning the regime only on the training set.

To ensure no look-ahead bias is introduced, we compute the threshold only on the training set. Then, we applied it to the whole dataset and split it into blocks. Afterwards, we trained two models, one for each regime, and applied the right one based on the rolling average of the standard deviation of each timestep, computed only on past values.

7 Hyperparameter Tuning Strategy

We perform a grid search over the following hyperparameters:

- EURGBP Forecast: Hidden_size $\in \{512, 1024, 2048\}$, nb_layers $\in \{1, 2, 4\}$, batch_size $\in \{256, 512\}$, n_epochs = 60
- NASDAQ100 Forecast: Hidden_size $\in \{512, 1024, 2048\}$, nb_layers $\in \{1, 2, 4\}$, batch_size = 128, n_epochs = 40.

While keeping the learning rate fixed at $1 \cdot 10^{-4}$, dropout at 0.1 (only if the network had more than one layer). The final model is selected based on the lowest validation error.

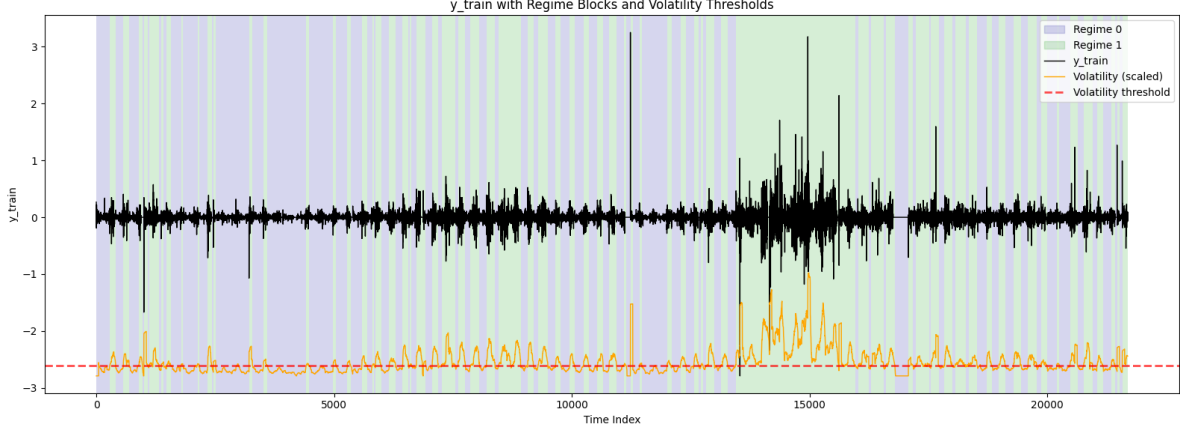


Figure 2: NASDAQ100 values, split by volatility regime

8 Results

8.1 EURGBP Forecast

We report the results of the baseline model alongside our LSTM models (standard and probabilistic) for 4 different metrics: Mean Absolute Error, (Root) Mean Square Error, and R^2 .

Model	batch_size	dropout	hidden_size	lr	n_epochs	nb_layers	validation loss
standard	512	0.1	1024	0.0001	60	2	0.055321
probabilistic	512	0	512	0.0001	60	1	-0.633442

Table 1: Best hyperparameters, EURGBP

Model	MAE	MSE	RMSE	R2
ARIMAX	0.2235	0.0957	0.3093	-0.0075
standard	0.166445	0.048683	0.220642	0.119293
probabilistic	0.165224	0.048054	0.219213	0.130666

Table 2: Comparison of Model Performance Metrics, EURGBP

We observe that the LSTM models perform better than the baseline for every metric. Comparing the two, instead, there is little difference in performance, with the probabilistic model having a slight edge.

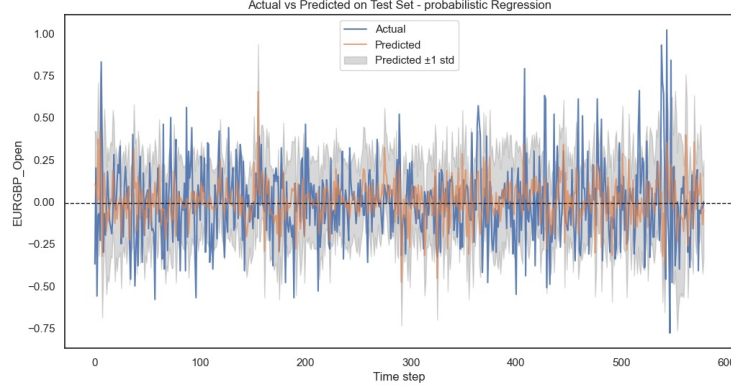


Figure 3: Predictions and actual values, EURGBP, Probabilistic model

8.2 NASDAQ100 Forecast

Model	batch_size	dropout	hidden_size	lr	n_epochs	nb_layers	validation loss
standard	128	0.0	512	0.0001	40	1	-0.633442
probabilistic	128	0.1	512	0.0001	40	4	0.055321

Table 3: Best hyperparameters, NASDAQ100

Model	MAE	MSE	RMSE	R2
standard	0.042551	0.003979	0.063083	-0.075801
probabilistic	0.040927	0.003740	0.063083	-0.011079

Table 4: Comparison of Model Performance Metrics, NASDAQ100

The results of the NASDAQ100 forecast shown in the table are quite unsatisfactory, and in particular, we remark a low R^2 , especially for datapoints belonging to the high-volatility regime. These suggest that the model is underfitted and predictions are very close to the mean, especially for the probabilistic model. Possible explanations include the very noisy nature of the data, resulting in a low signal-to-noise ratio, or the lack of important predictors.

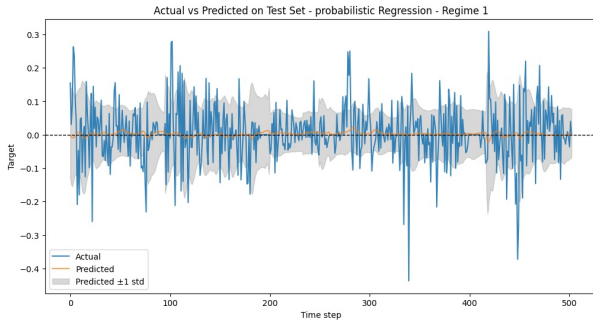


Figure 4: Probabilistic model - high volatility regime

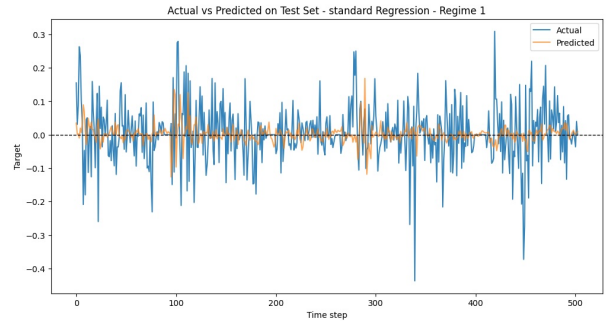


Figure 5: Standard model - high volatility regime

9 Attempt at a Trading Strategy

Given the results of our models, we decided to try to implement a simple trading strategy for EURGBP. We buy when the prediction of the return for the next day is above a threshold of 0.1, and sell if it is below -0.1 . We assume to close the trade the day after and 2 bips transaction costs. For the probabilistic model, we also ensured that the volatility of the prediction was below the mean of the ones on the training set (again, to avoid introducing biases).

While with the standard model the strategy is not particularly profitable, the probabilistic one, leveraging information about the risk of the trade, results in some profitability. Below is the result of the simulation.

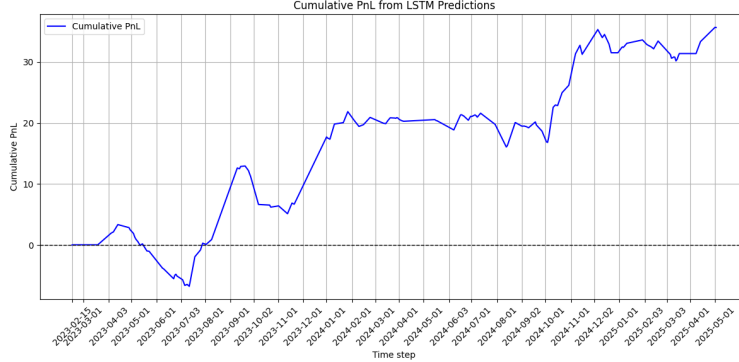


Figure 6: Profit&Losses, EURGBP, Probabilistic model

However, we are aware that the results presented here are based on historical data and simplified assumptions. In real-world scenarios, factors such as market impact, slippage, and changing market conditions can significantly affect performance. These results should not be interpreted as financial advice or as a recommendation for actual trading.

10 Model Limitations and Future Work

- **Conditions Shifts:** While > 20 years of daily data is substantial, regime shifts (e.g., Brexit, ECB policy changes) may introduce anomalies in the data. NLP models for sentiment analysis could mitigate this issue by intrinsically incorporating macroeconomic information [Hasan et al. \[2019\]](#).
- **Feature Selection:** Although we computed a large pool of indicators, an automatic feature-selection step (e.g., attention-based gating, or PCA) could improve fitting. In addition, some markets (e.g., emerging or Chinese markets) and asset classes (e.g., bonds) were not directly included in the analysis, potentially missing some key correlations.
- **Alternate Architectures:** Future experiments could compare against recent architectures such as the Temporal Fusion Transformer, which jointly handles static and temporal covariates with attention mechanisms [Bryan Lim and Pfister \[2020\]](#).

11 Conclusion

We presented a comprehensive pipeline for next-day EURGBP and next-5-minute NASDAQ100 forecasting that combines technical-indicator feature engineering, wavelet-based denoising, and LSTM modeling. Among the configurations tested, the Gaussian NLL loss used in a 'probabilistic' model achieved the best forecast performance and provided rather reliable predictive intervals. These findings lay the groundwork for more advanced architectures (e.g., [Bryan Lim and Pfister \[2020\]](#)) and for evaluating real-world trading impact.

References

- Yulei Rao Wei Bao, Jun Yue. A deep learning framework for financial time series using stacked autoencoders and long-short term memory, 2017.
- David L. Donoho. De-noising by soft-thresholding, 1995.
- Paolo Mercorelli Amin Karimi Dastgerdi. Investigating the effect of noise elimination on lstm models for financial markets prediction using kalman filter and wavelet transform, 2022.
- Md. Rakibul Hasan, Maisha Maliha, and M. Arifuzzaman. Sentiment analysis with nlp on twitter data. In *2019 International Conference on Computer, Communication, Chemical, Materials and Electronic Engineering (IC4ME2)*, pages 1–4, 2019. doi: 10.1109/IC4ME247184.2019.9036670.
- Nicolas Loeff Bryan Lim, Sercan O. Arık and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting, 2020.